

Labo 21

opleidingsonderdeel **Web Development 1**

studiegebied **Handelswetenschappen, bedrijfskunde**

Bachelor in de **toegepaste informatica**

campus **Kortrijk**

Sander Vandeputte

docent: **Pim Debaere**

academiejaar **2025-2026**

Demo storage

Demo 1 globale variabelen

Hier wordt enkel gebruik gemaakt van de globale variabelen. Bij het refreshen of heropenen van het tabblad wordt de teller niet onthouden. Het start dus elke keer terug vanaf 0	Verhoog Globale variabelen demo Klik op de knop om de teller te verhogen. De waarde van de teller is 5 .
--	---

Html

```
<!DOCTYPE html>
<html lang="nl">
  <head>
    <title>Globale variabelen</title>
  </head>
  <body>
    <p><input type="button" id="btnVerhoog" value="Verhoog"></p>
    <p>Globale variabelen demo</p>
    <p>Klik op de knop om de teller te verhogen.</p>
    <p id="txtResult">(nog niet geklikt)</p>
    <script src="../../scripts/demo%20storage/demo1.js"></script>
  </body>
</html>
```

Javascript

```
let teller=0; // de globale variabele voor onze teller

const setup =() => {
  let btnVerhoog=document.getElementById("btnVerhoog");
  btnVerhoog.addEventListener("click", verhoog);
};

const verhoog =() => {
  let txtResult=document.getElementById("txtResult");
  teller++;
  txtResult.innerHTML="De waarde van de teller is " + teller;
};

window.addEventListener("load", setup);
```

vragen demo 1

Blijft de data behouden als je de pagina herlaadt?

Nee. De data blijft niet behouden. De globale variabele wordt bij elke keer herladen opnieuw geïnitieerd.

Blijft de data behouden als je het tabblad sluit en de pagina opnieuw opent?

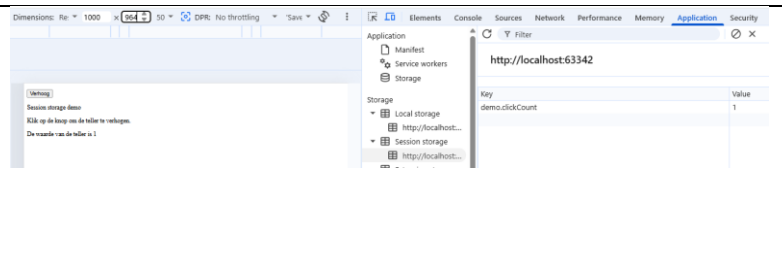
Nee, om dezelfde reden. De variabele bestaat enkel in het geheugen van de browser en verdwijnt wanneer het tabblad gesloten wordt.

Gebruiken twee tabbladen die dezelfde pagina tonen, dezelfde data?

Nee. Elk tabblad heeft zijn eigen geheugen en dus zijn eigen instantie van de globale variabele. Ze delen geen data

demo 2 session storage

Hier wordt de counter opgeslagen in de session storage. Bij het refreshen wordt dus de waarde onthouden. Als het tabblad afgesloten wordt, is de waarde weg.



Html

```
<!DOCTYPE html>
<html lang="nl">
  <head>
    <title>Session Storage</title>
  </head>
  <body>
    <p><input type="button" id="btnVerhoog" value="Verhoog"></p>
    <p>Session storage demo</p>
    <p>Klik op de knop om de teller te verhogen.</p>
    <p id="txtResult">(nog niet geklikt)</p>
    <script src="../../scripts/demo%20storage/demo2.js"></script>
  </body>
</html>
```

Javascript

```
const setup = () => {
  let btnVerhoog=document.getElementById("btnVerhoog");
  btnVerhoog.addEventListener("click", verhoog);
};

const verhoog = () => {
  let txtResult=document.getElementById("txtResult");
  let clickCount=0;
  if(typeof(Storage)=="undefined") {
    txtResult.innerHTML="Sorry, uw browser ondersteunt geen web
storage...";
  } else {
    clickCount=sessionStorage.getItem("demo.clickCount");
    if (clickCount) {
      clickCount=parseInt(clickCount)+1;
    } else {
      clickCount=1;
    }
    sessionStorage.setItem("demo.clickCount", clickCount.toString());
    txtResult.innerHTML="De waarde van de teller is " + clickCount;
  }
};

window.addEventListener("load", setup);
```

vragen demo 2

de data behouden als je de pagina herlaadt?

Ja. Session storage blijft behouden bij herladen

Blijft de data behouden als je het tabblad sluit en de pagina opnieuw opent?

Nee. Session storage is gekoppeld aan het tabblad en verdwijnt bij het sluiten ervan

Wordt de data gedeeld door verschillende tabbladen?

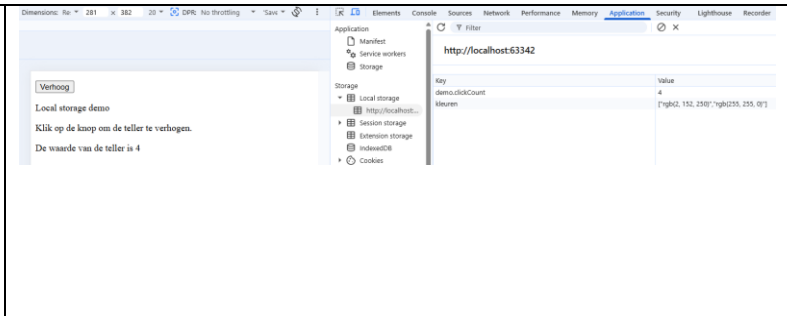
Nee. Elk tabblad heeft zijn eigen session storage.

Denk je dat een andere pagina aan de data geraakt in hetzelfde tabblad?

Nee. Wanneer je navigeert naar een andere pagina in hetzelfde tabblad wordt de session storage gewist. De nieuwe pagina start met een lege session storage.

demo 3 local storage

Hier wordt gebruik gemaakt van de local storage. Deze storage blijft bij wijze van spreken altijd bestaan totdat de gebruiker die zelf wist. Bij het refreshen of het sluiten van het tabblad worden deze waarden toch bewaard.



Html

```
<!DOCTYPE html>
<html lang="nl">
  <head>
    <title>Local Storage</title>
  </head>
  <body>
    <p><input type="button" id="btnVerhoog" value="Verhoog"></p>
    <p>Local storage demo</p>
    <p>Klik op de knop om de teller te verhogen.</p>
    <p id="txtResult">(nog niet geklikt)</p>
    <script src="../../scripts/demo%20storage/demo3.js"></script>
  </body>
</html>
```

Javascript

```
const setup = () => {
  let btnVerhoog=document.getElementById("btnVerhoog");
  btnVerhoog.addEventListener("click", verhoog);
}

const verhoog = () => {
  let txtResult=document.getElementById("txtResult");
  let clickCount=0;
  if(typeof(Storage)=="undefined") {
    txtResult.innerHTML="Sorry, uw browser ondersteunt geen web
storage...";
  } else {
    clickCount=localStorage.getItem("demo.clickCount");
    if (clickCount) {
      clickCount=parseInt(clickCount)+1;
    } else {
      clickCount=1;
    }
    localStorage.setItem("demo.clickCount", clickCount.toString());
    txtResult.innerHTML="De waarde van de teller is " + clickCount;
  }
}

window.addEventListener("load", setup);
```

vragen demo 3

Blijft de data behouden als je het tabblad sluit en de pagina opnieuw opent?

Ja. Local storage is niet gekoppeld aan een tabblad. De data blijft ongeacht of het tabblad sluit of opent.

Blijft de data behouden als je de pagina herlaadt?

Ja. Bij herladen wordt enkel de pagina opnieuw geladen, de local storage wordt hierbij niet aangeraakt.

Blijft de data behouden als je de browser afsluit, opnieuw start en de pagina weer opent?

Ja. Local storage is persistent en wordt opgeslagen op de harde schijf van de gebruiker. Het afsluiten van de browser heeft hier geen invloed op.

Blijft de data behouden als je de pagina in een andere browser opent?

Nee. Elke browser beheert zijn eigen local storage volledig onafhankelijk.

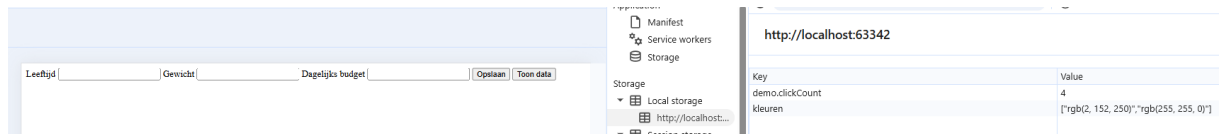
Wordt de data gedeeld door verschillende tabbladen als je de pagina meermaals opent?

Ja. Local storage is gekoppeld aan het domein en niet aan een tabblad. Alle tabbladen die dezelfde pagina (of hetzelfde domein) openen, hebben toegang tot dezelfde data

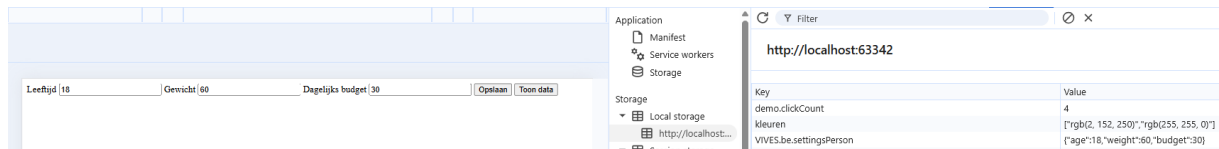
Denk je dat een andere pagina (andere file of URL) aan de data geraakt?

Ja. Local storage is afgebakend per domein, niet per pagina. Alle pagina's op hetzelfde domein delen dus dezelfde local storage, ongeacht welke file of URL je opent.

Demo storage JSON



Voor het ingeven van de testdata kan je zien dat er niks in de local storage aanwezig is.



Vanaf dat er op de knop “Opslaan” geklikt wordt, wordt de testdata in de local storage opgeslagen. Deze data is hier opgeslagen in een JSON string

Na het refreshen van de pagina, of na het sluiten en heropenen van het tabblad, worden de invulvelden automatisch gevuld met de eerder opgeslagen testdata wanneer je op "Toon data" klikt.

Html

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <title>Demo localStorage</title>
</head>
<body>
<form>
  <label for="txtAge">Leeftijd</label>
  <input id="txtAge" type="number">
  <label for="txtWeight">Gewicht</label>
  <input id="txtWeight" type="number">
  <label for="txtBudget">Dagelijks budget</label>
  <input id="txtBudget" type="number">
  <input type="button" id="btnSave" value="Opslaan">
  <input type="button" id="btnShow" value="Toon data">

  <script src="../../scripts/demo_storage_JSON/demo.js"></script>
</form>
</body>
</html>
```

Javascript

```
const setup = () => {
    document.getElementById("btnSave").addEventListener("click",
saveSettings);
    document.getElementById("btnShow").addEventListener("click",
showSettings);
};

const saveSettings = () => {
    let settings = {};
    let settingsJSON;

    // bouw settings object op basis van ingevulde gegevens
    // eigenlijk zouden we hier ook inputvalidering moeten doen
    settings.age = parseInt(document.getElementById("txtAge").value);
    settings.weight = parseInt(document.getElementById("txtWeight").value);
    settings.budget = parseInt(document.getElementById("txtBudget").value);

    // bewaar settings als JSON string in local storage
    settingsJSON = JSON.stringify(settings);
    localStorage.setItem("VIVES.be.settingsPerson", settingsJSON);
};

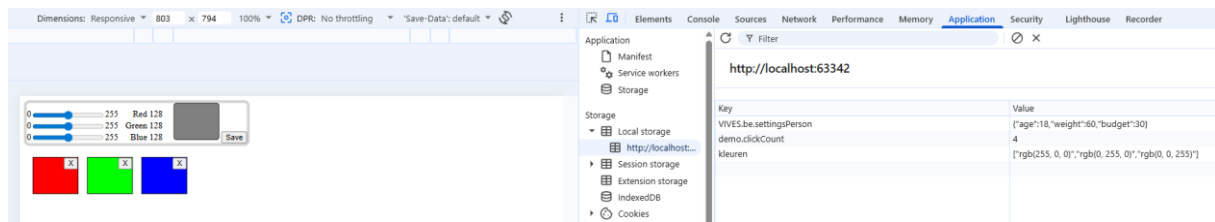
const showSettings = () => {
    let settings;
    let settingsJSON = localStorage.getItem("VIVES.be.settingsPerson");

    // Maak een leeg settings object, of bouw het op basis
    // van de settings JSON string in local storage
    if (settingsJSON === undefined) {
        settings = {
            age: 0,
            weight: 0,
            budget: 0
        };
    } else {
        settings = JSON.parse(settingsJSON);
    }

    // vul de invoervelden met de settings-waarden
    document.getElementById("txtAge").value = settings.age;
    document.getElementById("txtWeight").value = settings.weight;
    document.getElementById("txtBudget").value = settings.budget;
};

window.addEventListener("load", setup);
```

ColorPicker Pro



Html

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <title>Color</title>
  <link rel="stylesheet" href="../styles/colorpicker_PRO.css">
</head>

<body>
  <div class="colorPicker">
    <div class="components">
      <div class="component">
        <span>0<input type="range" id="sldRed" class="slider"
value="128" min="0" max="255">255</span>
        <span>Red</span>
        <span id="lblRed">0</span>
      </div>
      <div class="component">
        <span>0<input type="range" id="sldGreen" class="slider"
value="128" min="0" max="255">255</span>
        <span>Green</span>
        <span id="lblGreen">0</span>
      </div>
      <div class="component">
        <span>0<input type="range" id="sldBlue" class="slider"
value="128" min="0" max="255">255</span>
        <span>Blue</span>
        <span id="lblBlue">0</span>
      </div>
    </div>
    <div class="swatch rounded" id="swatch"></div>
    <input type="button" id="btnSave" value="Save">
  </div>
  <div id="swatchComponents"></div>
  <script src="../scripts/colorpicker_PRO.js"></script>
</body>
</html>
```

Javascript

```
const global = {
  huidigeKleur: "rgb(128, 128, 128)"
};

const initialize = () => {
  let opgeslagenKleuren = [];
  const opgeslagen = localStorage.getItem("kleuren");
  if (opgeslagen !== null) {
    opgeslagenKleuren = JSON.parse(opgeslagen);
  }
  opgeslagenKleuren.forEach(kleur => voegSwatchToe(kleur));

  document.querySelectorAll(".slider").forEach(slider => {
    slider.addEventListener("input", update);
  });

  document.getElementById("btnSave").addEventListener("click",
saveSwatch);

  update();
};

const saveSwatch = () => {
  voegSwatchToe(global.huidigeKleur);

  let opgeslagenKleuren = [];
  const opgeslagen = localStorage.getItem("kleuren");
  if (opgeslagen !== null) {
    opgeslagenKleuren = JSON.parse(opgeslagen);
  }

  opgeslagenKleuren.push(global.huidigeKleur);
  localStorage.setItem("kleuren", JSON.stringify(opgeslagenKleuren));
  console.log(opgeslagenKleuren);
};

const voegSwatchToe = (kleur) => {
  const [r, g, b] = kleur.replace("rgb(", "").replace(")", "").split(",");

  const swatch = document.createElement("div");
  swatch.className = "swatch";
  swatch.style.background = kleur;
  swatch.setAttribute("data-red", r);
  swatch.setAttribute("data-green", g);
  swatch.setAttribute("data-blue", b);

  swatch.addEventListener("click", setColorPickerFromSwatch);

  const btnDelete = document.createElement("input");
  btnDelete.setAttribute("type", "button");
  btnDelete.setAttribute("value", "X");
  btnDelete.addEventListener("click", deleteSwatch);

  swatch.appendChild(btnDelete);
  document.getElementById("swatchComponents").appendChild(swatch);
};

const setColorPickerFromSwatch = (event) => {
```

```

    const swatch = event.currentTarget;
    document.getElementById("sldRed").value = swatch.getAttribute("data-
red");
    document.getElementById("sldGreen").value = swatch.getAttribute("data-
green");
    document.getElementById("sldBlue").value = swatch.getAttribute("data-
blue");
    update();
};

const deleteSwatch = (event) => {
    event.stopPropagation();
    const swatch = event.target.parentNode;

    let opgeslagenKleuren = [];
    const opgeslagen = localStorage.getItem("kleuren");
    if (opgeslagen !== null) {
        opgeslagenKleuren = JSON.parse(opgeslagen);
    }

    const kleur = swatch.style.background;
    const index = opgeslagenKleuren.indexOf(kleur);
    if (index !== -1) opgeslagenKleuren.splice(index, 1);
    localStorage.setItem("kleuren", JSON.stringify(opgeslagenKleuren));
    console.log(opgeslagenKleuren);

    swatch.remove();
};

const update = () => {
    const red = document.getElementById("sldRed").value;
    const green = document.getElementById("sldGreen").value;
    const blue = document.getElementById("sldBlue").value;

    document.getElementById("lblRed").innerHTML = red;
    document.getElementById("lblGreen").innerHTML = green;
    document.getElementById("lblBlue").innerHTML = blue;

    const kleur = `rgb(${red}, ${green}, ${blue})`;
    document.getElementById("swatch").style.background = kleur;
    global.huidigeKleur = kleur;
};

window.addEventListener("load", initialize);

```

CSS

```
.slider {
  vertical-align:middle;
}

.component span:nth-of-type(2) {
  display:inline-block;
  width:3em;
  text-align:right;
}

.components {
  display:inline-block;
  height:100%;
}

.colorPicker {
  display:inline-block;
  border:solid 5px lightgrey;
  border-radius:10px;
  margin-bottom:20px;
}

.swatch {
  display:inline-block;
  border:solid 1px black;
  margin:0 0 0 1em;
  height:4em;
  width:5em;
}

.swatch.rounded {
  border-radius:5px;
}

.swatch>input {
  float:right;
}
```